

Metadata Database

The metadata database stores all information related to:

- Users
- Devices
- Files
- File versions
- File blocks

Please note that actual file contents are stored in cloud storage systems like Amazon S3, while the metadata database only stores metadata information.

Metadata Database Schema Design

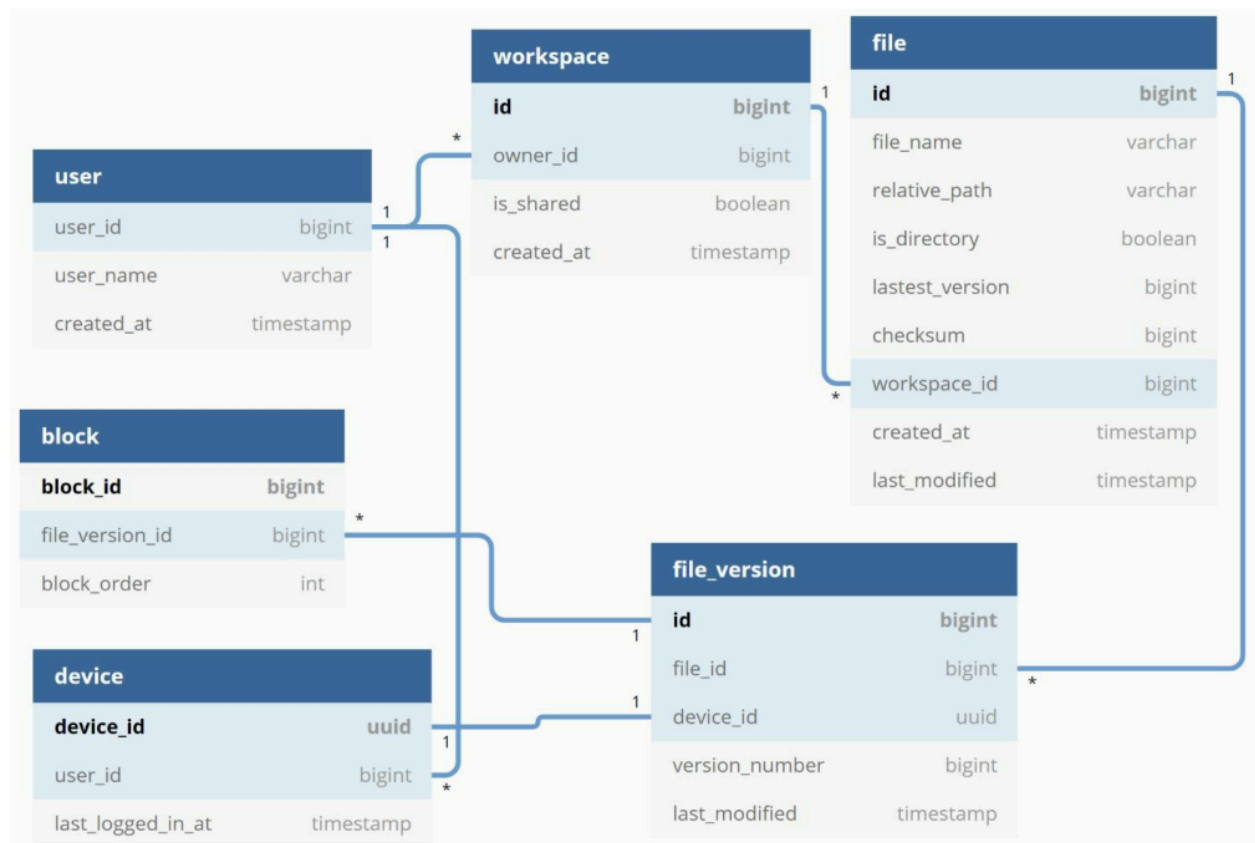


Figure shows a simplified database schema that contains the most important tables and fields used in the Google Drive system.

Important Tables

1. User Table

The **User** table stores basic information about users such as:

- Username
- Email
- Profile photo
- Account information

This table is used for authentication and user profile management.

2. Device Table

The **Device** table stores information about user devices.

A single user can use multiple devices, such as:

- Mobile phone
- Laptop
- Tablet

Important field:

- **push_id** → used for sending mobile push notifications.

3. Namespace Table

A namespace represents the root directory of a user.

Every user has an isolated namespace where all their files and folders are stored.

4. File Table

The **File** table stores information related to the latest version of a file, including:

- File name
- File path

- File size
- Upload status
- Latest version reference

This table is frequently accessed during uploads and downloads.

5. File_Version Table

The **File_Version** table stores revision history of files.

Important characteristics:

- Existing rows are read-only.
- Older versions are never modified.
- Ensures integrity of revision history.

This allows users to:

- View old versions
- Restore previous versions
- Track modifications

6. Block Table

The **Block** table stores information related to file blocks.

Each block contains:

- Block ID
- Hash value
- Storage location
- Block order

A file can be reconstructed by downloading and joining all blocks in the correct sequence.

Upload Flow

Now let us understand what happens internally when a client uploads a file.

To improve efficiency, the system performs two operations in parallel:

1. Add file metadata
2. Upload file blocks to cloud storage

File Upload Sequence Diagram

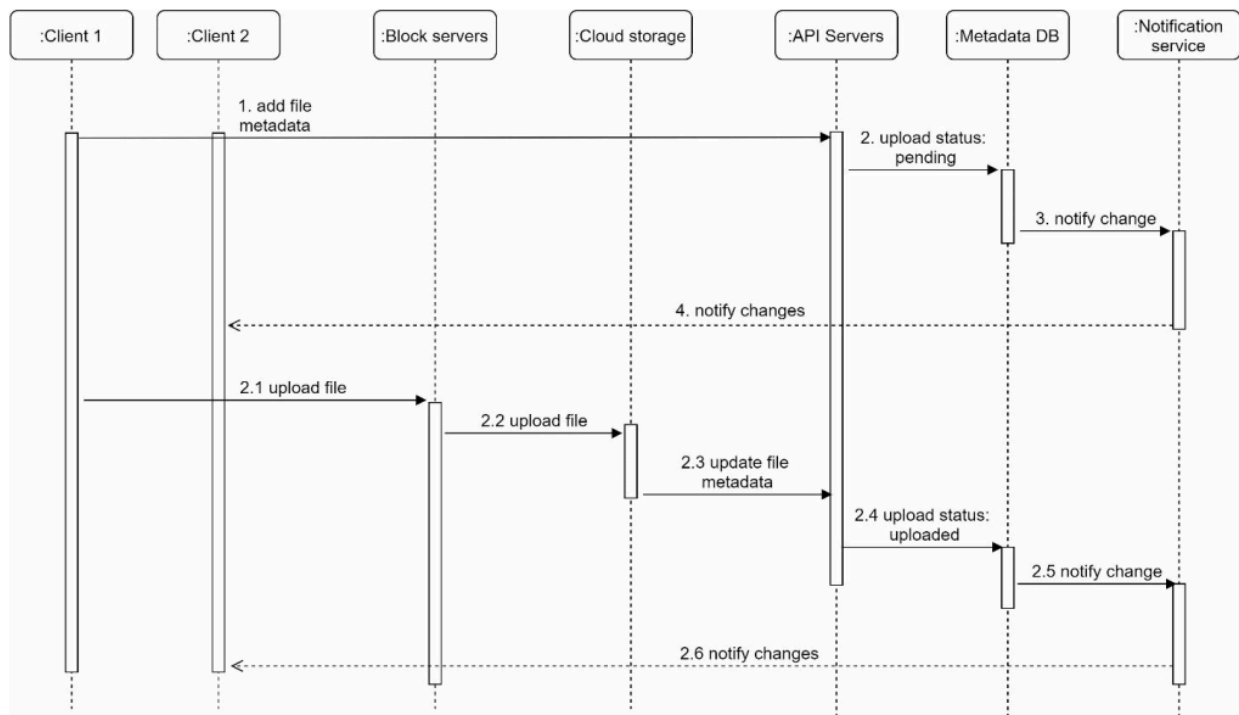


Figure illustrates the detailed upload workflow.

Both requests originate from **Client 1**.

A. Add File Metadata Flow

Step 1

Client 1 sends a request to API servers to add metadata of the new file.

The metadata includes:

- File name

- File size
- File type
- Owner information

Step 2

API servers store the metadata in the Metadata Database.

The file upload status is initially marked as:

pending

This indicates that metadata exists but file upload is not yet completed.

Step 3

API servers notify the Notification Service that a new file upload has started.

Step 4

Notification Service informs other relevant clients (for example, Client 2) that a file is currently being uploaded.

This ensures synchronization across multiple devices.

B. Upload File to Cloud Storage

Step 2.1

Client 1 uploads the file contents to Block Servers.

Step 2.2

Block Servers perform the following operations:

- Split file into blocks
- Compress blocks
- Encrypt blocks
- Upload blocks to cloud storage

This design improves:

- Scalability
- Upload speed
- Fault tolerance

Step 2.3

Once upload is complete, cloud storage triggers an upload completion callback.

The callback request is sent to API servers.

Step 2.4

API servers update the file upload status in Metadata DB from:

pending → uploaded

Step 2.5

API servers notify the Notification Service that file upload is complete.

Step 2.6

Notification Service informs other clients (Client 2) that the file is now fully uploaded and available.

File Edit Flow

When a file is edited, the workflow remains very similar.

The only difference is that:

- Only modified blocks are uploaded using delta sync.
- A new version entry is added into the `File_Version` table.

Download Flow

The download flow is triggered whenever:

- A file is added
- A file is modified
- A file is updated by another client

The client must first know that changes have occurred.

How Clients Detect File Changes

There are two scenarios:

Scenario 1: Client is Online

If Client A is online while another client modifies a file:

- Notification Service immediately informs Client A.
- Client A pulls the latest metadata and file blocks.

Scenario 2: Client is Offline

If Client A is offline during file modification:

- Updates are stored temporarily in offline backup storage/cache.
- When Client A reconnects, it fetches the latest changes.

Download Workflow

Once a client knows that updates exist, it performs the following operations.

File Download Flow

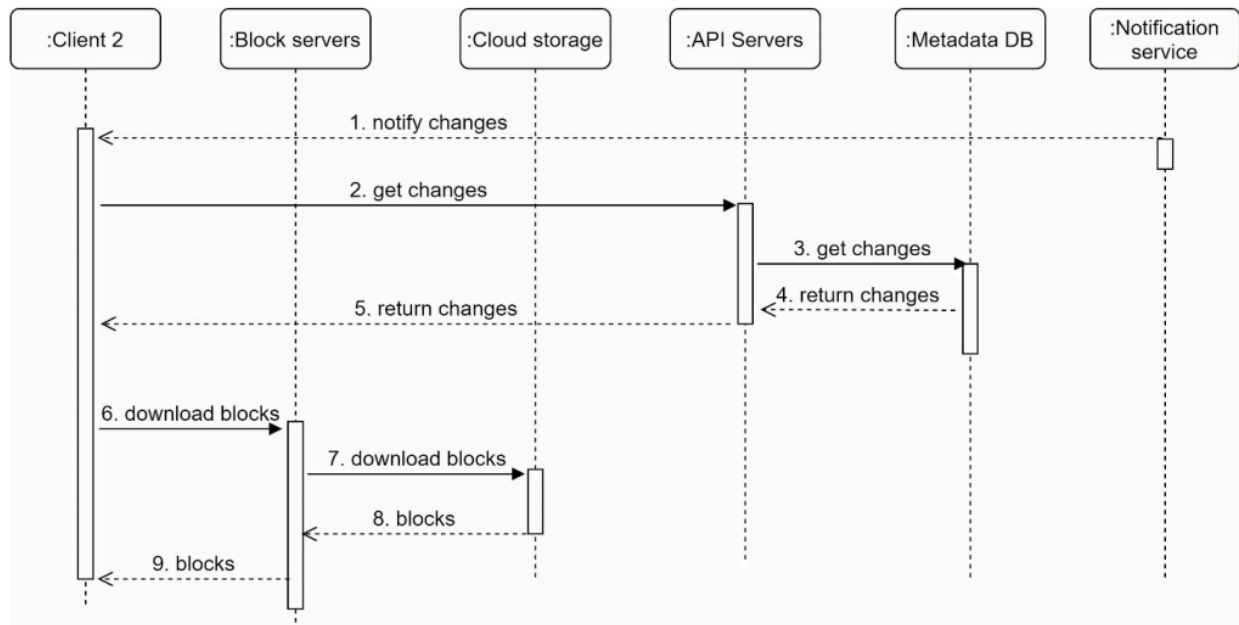


Figure illustrates the detailed download process.

Step 1

Notification Service informs Client 2 that a file has changed somewhere else.

Step 2

Client 2 sends a request to API servers to fetch updated metadata.

Step 3

API servers query the Metadata Database.

Step 4

Metadata Database returns updated metadata information.

Step 5

Client 2 receives the metadata response.

The metadata contains:

- File version
- Block identifiers
- Block order
- Storage locations

Step 6

Client 2 requests updated blocks from Block Servers.

Step 7

Block Servers download required blocks from Cloud Storage.

Step 8

Cloud Storage returns blocks to Block Servers.

Step 9

Client 2 downloads all updated blocks and reconstructs the complete file locally.

This architecture ensures:

- Efficient synchronization
- Minimal bandwidth usage
- Fast file updates
- Strong consistency across devices